FastAPI Notes

Module 1 — FastAPI Fundamentals

1. What is FastAPI?

FastAPI is a modern, high-performance Python web framework used to build **APIs and backend applications**.

It is built on top of:

- **Starlette** → handles web/HTTP functionality
- **Pydantic** → handles data validation and serialization
- **ASGI** → allows asynchronous web applications

FastAPI allows developers to create APIs using normal Python functions and type hints.

Simple example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}
```

Here:

- FastAPI() creates the application.
- @app.get("/") creates a GET endpoint.
- home() handles the request.
- The dictionary is automatically converted into a JSON response.

---

2. Why FastAPI?

FastAPI is popular because it provides:

High performance

It is designed for high-performance API development and uses **ASGI**.

Easy development

You can create an API with very little code.

Automatic validation

FastAPI uses **Pydantic** to validate incoming data.

Automatic documentation

FastAPI automatically generates:

- Swagger UI
- ReDoc
- OpenAPI schema

Type hints

FastAPI makes heavy use of Python type hints.

Example:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

The int tells FastAPI that user_id should be an integer.

---

3. Features of FastAPI

Important features include:

1. High performance

FastAPI is suitable for applications that need fast request processing.

2. Async support

It supports:

async def

which is useful for I/O operations such as:

- Database calls
- API calls

- File operations
- Network operations

3. Automatic data validation

FastAPI validates request data using Pydantic.

4. Automatic API documentation

Documentation is generated automatically.

5. Python type hints

Type hints are used for:

- Validation
- Documentation
- Editor support
- Request parsing

6. Dependency Injection

FastAPI provides a built-in dependency injection system.

It is commonly used for:

- Authentication
- Database sessions
- Common reusable logic

7. JSON support

FastAPI makes it easy to receive and return JSON data.

8. OpenAPI support

FastAPI automatically generates an OpenAPI specification for the API.

---

4. FastAPI vs Flask

Both FastAPI and Flask are Python web frameworks, but they have different design goals.

Feature	FastAPI	Flask

Feature	FastAPI	Flask
Type hints	Strongly used	Optional
Async support	Built-in	Supported, but not the core design
Performance	Generally higher for API workloads	Generally simpler/traditional
Validation	Automatic with Pydantic	Usually requires extensions/manual handling
API documentation	Automatic	Usually requires additional tools
Architecture	API-focused	General-purpose
Learning curve	Easy	Very easy
ASGI	Yes	Traditionally WSGI; newer Flask versions support async views but deployment model differs

Simple difference

Flask: Minimal and flexible web framework.

FastAPI: Modern framework focused heavily on building APIs using Python type hints, validation, async support, and automatic documentation.

---

5. FastAPI vs Django

FastAPI and Django serve different purposes.

Feature	FastAPI	Django
Main focus	APIs	Full web applications
Architecture	Lightweight	Batteries-included
ORM	Not built in	Built-in ORM
Admin panel	Not built in	Built-in
Authentication	Usually added/configured	Many built-in facilities
API documentation	Automatic	Usually requires additional tools
Flexibility	High	Structured
Learning	Relatively simple	Larger framework to learn

Simple understanding

FastAPI is a strong choice when the main requirement is building APIs/backend services.

Django is useful when you need a complete web framework with features such as:

- ORM
- Admin panel
- Authentication
- Templates
- Forms
- Middleware

---

6. Advantages and Disadvantages

Advantages

1. High performance

ASGI and async capabilities make FastAPI suitable for high-performance APIs.

2. Automatic validation

Incoming data can be validated automatically.

3. Automatic documentation

Swagger UI and ReDoc are generated automatically.

4. Developer productivity

Type hints, validation, documentation, and editor support reduce development effort.

5. Async support

Useful for applications involving many I/O operations.

6. Easy to learn

Developers familiar with Python can start quickly.

Disadvantages

1. Smaller ecosystem than Django

Django has a very mature ecosystem with many built-in components.

2. More decisions for the developer

FastAPI intentionally does not provide everything.

For example, you may need to choose:

- ORM
- Authentication library
- Database tools
- Project architecture

3. Async can be misunderstood

Using async does not automatically make every operation faster.

Async is particularly useful for **I/O-bound operations**.

4. Large applications need proper architecture

As the project grows, you need to organize:

- Routers
- Services
- Models
- Schemas
- Dependencies
- Database logic

---

7. ASGI vs WSGI

This is an important backend concept.

WSGI

WSGI = Web Server Gateway Interface

It is a standard interface between a Python web application and a web server.

Traditional Python frameworks such as Flask and Django historically used WSGI.

WSGI is primarily designed around **synchronous request handling**.

ASGI

ASGI = Asynchronous Server Gateway Interface

It is a newer standard designed to support:

- Synchronous applications
- Asynchronous applications
- WebSockets
- Long-lived connections

FastAPI uses ASGI.

Simple difference

WSGI
↓
Traditional synchronous Python web applications
ASGI
↓
Modern synchronous + asynchronous Python applications

Why FastAPI uses ASGI?

Because FastAPI supports asynchronous programming and modern communication patterns such as WebSockets.

---

8. Uvicorn

Uvicorn is an ASGI web server.

FastAPI itself is the web framework.

Uvicorn is responsible for **running the FastAPI application and communicating with clients through HTTP**.

Think of it like:

Client
↓
Uvicorn
↓
FastAPI
↓

Your Python code

Example:

uvicorn main:app

Here:

main → main.py
app → FastAPI application object

So:

uvicorn main:app

means:

Start the app object from main.py.

---

9. Starlette

Starlette is a lightweight ASGI framework/toolkit used by FastAPI for web-related functionality.

It provides functionality such as:

- Routing
- Middleware
- Request handling
- Response handling
- WebSockets
- Background tasks

FastAPI adds additional features on top of this foundation.

A simplified relationship is:

FastAPI
↓
Starlette
↓
ASGI

---

10. Pydantic

Pydantic is a Python library used for data validation and parsing.

FastAPI uses Pydantic extensively.

For example:

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
```

This describes the expected structure of a user.

If the API receives:

```
{
    "name": "Prathamesh",
    "age": 22
}
```

Pydantic validates the data according to the model.

Pydantic is mainly used for

- Request validation
- Response validation/serialization
- Data parsing
- Defining schemas

---

11. How FastAPI Works Internally

A simplified architecture looks like this:

```
Client
  ↓
HTTP Request
  ↓
Uvicorn
  ↓
ASGI
  ↓
```

FastAPI
↓
Starlette
↓
Routing
↓
Path Operation
↓
Dependency Injection
↓
Pydantic Validation
↓
Your Function
↓
Response
↓
Uvicorn
↓
Client

FastAPI uses Python type information and Pydantic models to understand the expected input and output.

For example:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"id": user_id}
```

FastAPI knows that:

/user/10 → valid integer
/user/abc → invalid according to int requirement

---

12. FastAPI Request-Response Lifecycle

When a client sends a request:

GET /users/10

the general flow is:

Step 1 — Client sends request

Browser, frontend, Postman, mobile app, etc. sends an HTTP request.

Step 2 — Uvicorn receives request

Uvicorn acts as the ASGI server.

Step 3 — FastAPI receives request

FastAPI processes the ASGI request.

Step 4 — Routing

FastAPI determines which route matches:

/users/{user_id}

Step 5 — Parameter extraction

FastAPI extracts:

user_id = 10

Step 6 — Validation

FastAPI/Pydantic validates the data.

Step 7 — Dependency execution

If the endpoint has dependencies, FastAPI resolves them.

Step 8 — Path operation function executes

```
def get_user(user_id: int):
    ...
```

Step 9 — Response creation

The returned Python object is converted into an appropriate HTTP response, commonly JSON.

Step 10 — Response sent to client

FastAPI
↓
Uvicorn
↓
Client

---

13. FastAPI Project Structure

A small project can start with:

```
my_project/
├── main.py
├── requirements.txt
└── .env
```

For a larger application:

```
my_project/
├── app/
│   ├── main.py
│   ├── routers/
│   │   ├── users.py
│   │   └── products.py
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── dependencies/
│   └── database/
├── tests/
├── .env
├── .gitignore
└── requirements.txt
```

Common responsibilities

main.py

Application entry point.

routers/

Contains API routes.

schemas/

Contains Pydantic models.

models/

Usually contains database models.

services/

Contains business logic.

database/

Contains database connection/configuration.

tests/

Contains automated tests.

---

14. Installing FastAPI and Uvicorn

Create a virtual environment:

python -m venv venv

Activate it on Windows:

venv\Scripts\activate

Install packages:

pip install fastapi uvicorn

Verify:

pip show fastapi
pip show uvicorn

---

15. Running a FastAPI Application

Suppose your file is:

main.py

and contains:

from fastapi import FastAPI

```
app = FastAPI()

@app.get("/")
def home():
   return {"message": "Hello"}
```

Run:

```
uvicorn main:app
```

The syntax is:

```
uvicorn <module>:<application_object>
```

So:

```
main:app
```

means:

main.py → app

Module 2 — First FastAPI Application

1. Creating FastAPI() Application

First import FastAPI:

```
from fastapi import FastAPI
```

Then create the application:

```
app = FastAPI()
```

app is the main FastAPI application object.

Routes are registered using this object.

Example:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
```

```
def home():
    return {"message": "Hello World"}
```

---

2. Application Instance

This:

app = FastAPI()

creates an **instance of the FastAPI application**.

The instance manages things such as:

- Routes
- Middleware
- Dependencies
- OpenAPI configuration
- Application behavior

The variable name does not have to be app.

For example:

application = FastAPI()

Then the Uvicorn command becomes:

uvicorn main:application

---

3. @app.get()

@app.get() creates a **GET route**.

Example:

```
@app.get("/")
def home():
    return {"message": "Home Page"}
```

When the client sends:

GET /

FastAPI calls:

home()

GET is generally used for

- Retrieving data
- Reading resources
- Searching/querying resources

---

4. @app.post()

POST is generally used to **create a new resource or submit data**.

Example:

```
@app.post("/users")
def create_user():
    return {"message": "User created"}
```

Request:

POST /users

---

5. @app.put()

PUT is generally used to **update/replace an existing resource**.

Example:

```
@app.put("/users/{user_id}")
def update_user(user_id: int):
    return {"message": f"User {user_id} updated"}
```

---

6. @app.patch()

PATCH is generally used for a **partial update**.

For example, a user has:

```
{
    "name": "Rahul",
    "age": 25,
    "city": "Pune"
```

}

If you only want to change the city, PATCH can update only that field.

PATCH /users/10

PUT vs PATCH

PUT

Usually represents replacing/updating the resource as a whole.

PATCH

Usually represents updating only specific fields.

---

7. @app.delete()

DELETE is used to remove a resource.

Example:

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    return {"message": f"User {user_id} deleted"}
```

Request:

DELETE /users/10

---

8. Route / Path Operation

A **route** defines the URL and HTTP method that an API supports.

Example:

```
@app.get("/users")
def get_users():
    return {"users": []}
```

Here:

GET → HTTP method

/users → path

Together they define an API operation.

FastAPI documentation often calls this a **path operation**.

---

9. Path Operation Function

The function associated with a route is called the **path operation function**.

Example:

```
@app.get("/users")
def get_users():
    return {"users": []}
```

Here:

get_users()

is the path operation function.

It contains the logic that should execute when the corresponding request arrives.

---

10. Uvicorn

To run the application:

uvicorn main:app

If the application is in:

main.py

and the application object is:

app = FastAPI()

then:

main:app

connects Uvicorn to the FastAPI application.

---

11. --reload

During development, use:

uvicorn main:app --reload

--reload automatically restarts the development server when code changes are detected.

Without reload

You usually need to stop and restart the server after making changes.

With reload

Change code
↓
Uvicorn detects change
↓
Server reloads
↓
New code runs

Important: --reload is primarily a **development feature**, not something normally used for production deployment.

---

12. Host and Port

You can specify the host and port:

uvicorn main:app --host 127.0.0.1 --port 8000

Host

The host determines which network interface the server listens on.

For local development:

127.0.0.1

means localhost.

Port

The port identifies the network port.

Common FastAPI development port:

8000

Therefore:

http://127.0.0.1:8000

is the local API address.

---

13. Swagger UI

FastAPI automatically provides interactive API documentation through **Swagger UI**.

By default:

/docs

So if your application runs at:

http://127.0.0.1:8000

Swagger UI is available at:

http://127.0.0.1:8000/docs

Swagger UI allows you to:

- See available endpoints
- See HTTP methods
- See parameters
- See request schemas
- Execute API requests directly
- View responses

It is extremely useful during API development and testing.

---

14. ReDoc

FastAPI also provides **ReDoc** documentation.

Default URL:

/redoc

Example:

http://127.0.0.1:8000/redoc

ReDoc provides another interface for viewing the API documentation.

Swagger vs ReDoc

Swagger UI
→ Interactive API testing/documentation
ReDoc
→ Clean API documentation interface

Both are generated from the same OpenAPI specification.

---

15. OpenAPI Documentation

OpenAPI is a standard specification for describing APIs.

FastAPI automatically generates an OpenAPI schema for your application.

By default:

/openapi.json

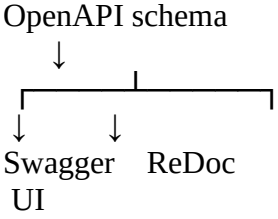
Example:

http://127.0.0.1:8000/openapi.json

The OpenAPI schema describes things such as:

- Available endpoints
- HTTP methods
- Parameters
- Request bodies
- Response structures
- Validation information

FastAPI uses this OpenAPI schema to generate:



So Swagger UI and ReDoc are essentially **documentation interfaces built from the OpenAPI schema**.

Module 3 — HTTP Fundamentals

HTTP fundamentals are important because FastAPI is primarily used to build **HTTP APIs**.

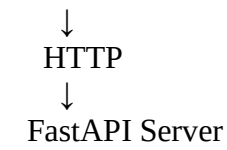
---

1. What is HTTP?

HTTP = HyperText Transfer Protocol

HTTP is a communication protocol used for communication between a **client** and a **server**.

For example:



HTTP defines how a client should:

- Send a request
- Specify what it wants
- Provide data
- Receive a response

Example:

GET /users

means the client is requesting the /users resource.

---

2. Client and Server

Client

A **client** is the application that sends a request.

Examples:

- Web browser
- React application
- Mobile application
- Postman
- Another backend service

Server

A **server** receives the request, processes it, and sends a response.

In a FastAPI application:

Client
↓
Uvicorn
↓
FastAPI
↓
Database / Business Logic

Example

A React frontend requests:

GET /users

FastAPI processes the request and returns the users.

---

3. HTTP Request

An **HTTP request** is a message sent by a client to a server.

A request can contain:

HTTP Method
URL

Headers
Query Parameters
Path Parameters
Request Body

Example:

```
POST /users
Content-Type: application/json

{
   "name": "Prathamesh",
   "age": 22
}
```

Here:

- POST → HTTP method
- /users → path
- Content-Type → header
- JSON object → request body

---

4. HTTP Response

An **HTTP response** is the message sent by the server back to the client.

A response generally contains:

Status Code
Headers
Response Body

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
   "message": "User found"
}
```

Here:

- 200 → status code

- Content-Type → response header
- JSON → response body

---

5. HTTP Methods

HTTP methods tell the server **what operation the client wants to perform**.

The most commonly used methods in REST APIs are:

GET
POST
PUT
PATCH
DELETE

GET

Used to **retrieve/read data**.

Example:

GET /users

Meaning:

Give me the users.

FastAPI:

```
@app.get("/users")
def get_users():
    return {"users": []}
```

GET requests generally should not be used to modify server data.

POST

Used to **create a new resource** or submit data.

Example:

POST /users

Request body:

```
{
   "name": "Rahul",
   "age": 25
}
```

FastAPI:

```
@app.post("/users")
def create_user():
   return {"message": "User created"}
```

PUT

Used to **replace or completely update a resource**.

Example:

PUT /users/10

The request may contain the complete updated resource.

Conceptually:

```
Existing User
    ↓
Complete replacement
    ↓
Updated User
```

PATCH

Used for a **partial update**.

Suppose a user has:

```
{
   "name": "Rahul",
   "age": 25,
   "city": "Pune"
}
```

If only the city needs to change:

```
PATCH /users/10
{
   "city": "Mumbai"
}
```

Only the specified field needs to be updated.

PUT vs PATCH

PUT
→ Usually complete replacement/update

PATCH
→ Partial update

DELETE

Used to delete a resource.

Example:

DELETE /users/10

Meaning:

Delete user 10.

FastAPI:

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
   return {"message": "User deleted"}
```

---

6. HTTP Status Codes

HTTP status codes tell the client **what happened to the request**.

They are divided into categories:

1xx → Informational
2xx → Success
3xx → Redirection

4xx → Client error
5xx → Server error

---

200 — OK

Means the request was successful.

Commonly used for:

GET
PUT
PATCH

Example:

```
{
   "message": "User retrieved successfully"
}
```

---

201 — Created

Means the request successfully created a new resource.

Commonly used with:

POST

Example:

```
POST /users
      ↓
User created
      ↓
201 Created
```

---

204 — No Content

Means the request was successful, but there is **no response body**.

Often used when deleting a resource.

```
DELETE /users/10
      ↓
```

204 No Content

4xx — Client Errors

A 4xx status code generally means there is a problem with the client's request.

---

400 — Bad Request

The server cannot process the request because the request is invalid.

Example:

Invalid request format
Invalid data
Malformed request

Think:

The client sent a bad request.

---

401 — Unauthorized

Means the request requires authentication or the provided authentication credentials are invalid.

Example:

Client
↓
GET /profile
↓
No valid authentication
↓
401

Important distinction:

401 → Authentication problem

---

403 — Forbidden

The server understands the request but refuses to allow it.

Usually associated with **authorization/permission** problems.

Example:

User → Admin endpoint
↓
User is authenticated
but does not have permission
↓
403 Forbidden

401 vs 403

401 → Who are you? / Authentication problem

403 → I know who you are, but you don't have permission.

---

404 — Not Found

The requested resource does not exist.

Example:

GET /users/9999

If user 9999 doesn't exist:

404 Not Found

---

409 — Conflict

The request conflicts with the current state of the resource.

Common example:

Trying to register an email that already exists.

POST /users
email = abc@gmail.com
↓
Email already exists
↓

409 Conflict

---

422 — Unprocessable Content

In FastAPI, 422 is commonly associated with **validation errors**.

Example:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    ...
```

Request:

/users/abc

FastAPI expects:

user_id → integer

but receives:

abc → string

FastAPI can return a validation error response.

In current HTTP terminology, this status is commonly described as **422 Unprocessable Content**. You may also see older documentation calling it **422 Unprocessable Entity**.

---

500 — Internal Server Error

Means something went wrong on the server while processing the request.

Example:

```
Client
  ↓
Request
  ↓
FastAPI
  ↓
Unexpected exception
  ↓
```

500 Internal Server Error

This generally indicates a **server-side problem**, not a normal validation problem from the client.

---

7. HTTP Headers

HTTP headers contain additional information about a request or response.

They provide metadata.

Example:

Content-Type: application/json
Authorization: Bearer <token>
Accept: application/json

Common headers

Content-Type

Tells the receiver what type of data is being sent.

Authorization

Contains authentication credentials/token information.

Accept

Tells the server what response format the client prefers.

Request headers

Sent by the client.

Response headers

Sent by the server.

---

8. Content-Type

Content-Type tells the receiver **what format the request or response body uses**.

Example:

Content-Type: application/json

means the body contains JSON.

Other examples:

application/json
text/html
text/plain
multipart/form-data
application/x-www-form-urlencoded

For a FastAPI JSON API, the most common content type is:

application/json

---

9. JSON

JSON = JavaScript Object Notation

JSON is a lightweight text format commonly used to exchange data between clients and servers.

Example:

```
{
   "name": "Prathamesh",
   "age": 22,
   "skills": ["Python", "FastAPI", "SQL"]
}
```

JSON supports values such as:

String
Number
Boolean
Array
Object
null

FastAPI commonly receives and returns JSON data.

---

10. REST API

REST = Representational State Transfer

REST is an architectural style for designing network APIs.

A REST API typically represents things as **resources**.

For example:

/users
/products
/orders

Instead of creating separate action-based URLs such as:

/createUser
/deleteUser
/updateUser

REST commonly uses HTTP methods to describe the operation:

GET /users → Get users
POST /users → Create user
GET /users/10 → Get user 10
PUT /users/10 → Update user 10
DELETE /users/10 → Delete user 10

---

11. RESTful API Principles

A RESTful API generally follows several important principles.

1. Client-Server Separation

Client and server have separate responsibilities.

Frontend
↕
API
↕
Backend

The frontend handles the user interface.

The backend handles business logic and data.

---

2. Statelessness

Each request should contain the information needed to process it.

The server should not depend on remembering the previous request in order to understand the current request.

For example, authentication information is commonly sent with each request through a token.

Request 1 → authentication information
Request 2 → authentication information
Request 3 → authentication information

---

3. Resource-Based URLs

URLs represent resources.

Good REST-style examples:

/users
/products
/orders
/users/10
/products/25

---

4. Use HTTP Methods Correctly

HTTP methods communicate the intended operation:

GET → Read
POST → Create
PUT → Replace/update
PATCH → Partial update
DELETE → Delete

---

5. Representation

A resource can be represented in a format such as JSON.

Example:

```
{
   "id": 10,
   "name": "Prathamesh"
}
```

Module 4 — Path Parameters

1. What are Path Parameters?

A **path parameter** is a variable value included directly inside the URL path.

Example:

/users/10

Here:

10

is the user ID.

We define it using {}:

/users/{user_id}

Path parameters are useful when the value identifies a **specific resource**.

---

2. Syntax

FastAPI syntax:

```
@app.get("/users/{user_id}")
def get_user(user_id):
   return {"user_id": user_id}
```

When the client requests:

/users/10

FastAPI passes:

user_id = 10

to the function.

---

3. Type Declaration

You can specify the type of the path parameter:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

Here:

user_id: int

means FastAPI expects an integer.

This provides:

- Validation
- Type conversion
- Better documentation
- Editor support

---

4. Multiple Path Parameters

An endpoint can contain multiple path parameters.

Example:

```
@app.get("/users/{user_id}/orders/{order_id}")
def get_order(user_id: int, order_id: int):
    return {
        "user_id": user_id,
        "order_id": order_id
    }
```

Request:

/users/10/orders/500

FastAPI extracts:

user_id = 10
order_id = 500

---

5. Path Parameter Validation

FastAPI validates path parameters according to their declared types.

Example:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    ...
```

Valid:

/users/10

Invalid:

/users/abc

because:

abc ≠ integer

FastAPI returns a validation error instead of passing invalid data to the function.

---

6. Automatic Type Conversion

FastAPI can convert compatible path parameter values into the declared Python type.

Example:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {
        "id": user_id,
        "type": str(type(user_id))
    }
```

Request:

/users/10

FastAPI converts the incoming path value into:

10

which is a Python int.

So HTTP URL values are initially text-like, but FastAPI uses the type declaration to parse and validate them.

---

7. Path()

Path() allows you to provide additional configuration and validation for a path parameter.

Example:

```
from fastapi import Path

@app.get("/users/{user_id}")
def get_user(
    user_id: int = Path(...)
):
    return {"user_id": user_id}
```

Path() can be used to specify things such as:

- Description
- Constraints
- Metadata
- Validation rules

---

8. Path Parameter Constraints

You can add restrictions to a path parameter.

For example, suppose user_id must be greater than 0.

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(
```

```
    user_id: int = Path(..., gt=0)
):
    return {"user_id": user_id}
```

Here:

gt=0

means:

user_id > 0

Other useful constraints include:

gt → greater than
ge → greater than or equal
lt → less than
le → less than or equal

---

9. Path Parameter Documentation

You can add descriptions using Path().

Example:

```
from fastapi import Path

@app.get("/users/{user_id}")
def get_user(
    user_id: int = Path(
        ...,
        description="The unique ID of the user"
    )
):
    return {"user_id": user_id}
```

This information can appear automatically in the API documentation.

Module 5 — Query Parameters

1. What are Query Parameters?

Query parameters are optional or required values passed after ? in the URL.

Example:

/users?limit=10

Here:

limit=10

is a query parameter.

Multiple query parameters:

/users?limit=10&active=true

Here:

limit=10
active=true

are query parameters.

---

2. Path Parameter vs Query Parameter

Path parameter

/users/10

Used to identify a specific resource.

10 → user_id

Query parameter

/users?limit=10

Used to provide additional options/filtering.

limit → query parameter

A useful mental model:

/users/{user_id}
↑
Path parameter

```
/users?limit=10
      ↑
   Query parameter
```

---

3. Required Query Parameters

A query parameter can be required.

Example:

```
@app.get("/users")
def get_users(limit: int):
    return {"limit": limit}
```

Now:

/users?limit=10

is valid.

But:

/users

does not provide limit.

FastAPI reports a validation error because the required parameter is missing.

---

4. Optional Query Parameters

Use a default value or an optional type when a query parameter isn't required.

Example:

```
@app.get("/users")
def get_users(limit: int | None = None):
    return {"limit": limit}
```

Now both are valid:

/users

and:

/users?limit=10

If the parameter isn't provided:

limit = None

---

5. Default Values

You can provide a default value.

```
@app.get("/users")
def get_users(limit: int = 10):
    return {"limit": limit}
```

If the request is:

/users

then:

limit = 10

If:

/users?limit=20

then:

limit = 20

---

6. Type Conversion

FastAPI uses the type declaration to parse and validate query parameters.

Example:

```
@app.get("/users")
def get_users(limit: int):
    return {"limit": limit}
```

Request:

/users?limit=10

FastAPI converts the value into an integer.

Conceptually:

URL value
  ↓
"10"
  ↓
FastAPI/Pydantic
  ↓
10

---

7. Boolean Query Parameters

FastAPI supports boolean query parameters.

Example:

```
@app.get("/users")
def get_users(active: bool = True):
    return {"active": active}
```

Request:

/users?active=false

FastAPI parses the value as a Python boolean.

active = False

This is useful for filters such as:

active
verified
available
is_admin

---

8. Multiple Query Parameters

You can define multiple query parameters.

```
@app.get("/users")
def get_users(
    limit: int = 10,
```

```
    skip: int = 0,
    active: bool = True
):
    return {
        "limit": limit,
        "skip": skip,
        "active": active
    }
```

Request:

/users?limit=20&skip=10&active=false

FastAPI extracts all three values.

---

9. Query Parameter Validation

FastAPI can validate query parameters according to their types and constraints.

Example:

```
@app.get("/users")
def get_users(limit: int):
    return {"limit": limit}
```

This ensures limit should be an integer.

You can also add constraints using Query().

---

10. Query()

Query() provides additional configuration and validation for query parameters.

Example:

```
from fastapi import Query

@app.get("/users")
def get_users(
    limit: int = Query(10)
):
    return {"limit": limit}
```

You can use Query() for:

- Validation
- Default values
- Descriptions
- String length constraints
- Numeric constraints
- Documentation metadata

---

11. min_length

min_length specifies the minimum number of characters for a string.

Example:

```
from fastapi import Query

@app.get("/users")
def search_users(
    name: str = Query(..., min_length=3)
):
    return {"name": name}
```

Valid:

/users?name=Raj

Invalid:

/users?name=Ra

because the minimum length is 3.

---

12. max_length

max_length specifies the maximum number of characters.

name: str = Query(..., max_length=20)

The value cannot contain more than 20 characters.

---

13. gt

gt means:

Greater Than

Example:

limit: int = Query(10, gt=0)

Requirement:

limit > 0

Therefore:

limit=10 → valid
limit=1 → valid
limit=0 → invalid

---

14. ge

ge means:

Greater Than or Equal

Example:

age: int = Query(..., ge=18)

Requirement:

age >= 18

So:

18 → valid
19 → valid
17 → invalid

---

15. lt

lt means:

Less Than

Example:

age: int = Query(..., lt=100)

Requirement:

age < 100

Therefore:

99 → valid
100 → invalid

---

16. le

le means:

Less Than or Equal

Example:

age: int = Query(..., le=100)

Requirement:

age <= 100

Therefore:

100 → valid
101 → invalid

Module 6 — Request Body

1. What is Request Body?

The **request body** contains data sent by the client to the server.

It is mainly used with:

- POST
- PUT
- PATCH

Example:

```
{
  "name": "Prathamesh",
  "age": 22
}
```

Here, the JSON data is the **request body**.

Interview Point

Request body is data sent from the client to the server, usually in JSON format.

---

2. JSON Request Body

JSON is the most common format for sending request body data in REST APIs.

Example:

```
{
  "name": "John",
  "email": "john@gmail.com",
  "age": 25
}
```

FastAPI can automatically parse JSON data received from the client.

---

3. Request Body Using Pydantic

FastAPI commonly uses **Pydantic models** to define the structure of request data.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
```

```
    return user
```

The client sends:

```
{
  "name": "John",
  "age": 25
}
```

FastAPI:

1. Receives JSON
2. Converts it into a Python object
3. Validates the data
4. Passes it to the function

Interview Point

Pydantic models define and validate the expected structure of request bodies.

---

4. Single Body Parameter

A single Pydantic model can represent the complete request body.

```
class Product(BaseModel):
    name: str
    price: float

@app.post("/products")
def create_product(product: Product):
    return product
```

Request:

```
{
  "name": "Laptop",
  "price": 50000
}
```

Here, product is the single body parameter.

---

5. Multiple Body Parameters

FastAPI can accept multiple Pydantic models in one request.

```
class User(BaseModel):
    name: str

class Address(BaseModel):
    city: str

@app.post("/users")
def create_user(user: User, address: Address):
    return {
        "user": user,
        "address": address
    }
```

Request:

```
{
  "user": {
    "name": "John"
  },
  "address": {
    "city": "Pune"
  }
}
```

Interview Point

Multiple body parameters are represented as separate fields in the JSON request.

---

6. Nested Request Bodies

A Pydantic model can contain another Pydantic model.

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    address: Address
```

Request:

```
{
  "name": "John",
  "address": {
    "city": "Pune",
    "pincode": 411001
  }
}
```

This is called a **nested request body**.

Real-world Example

```
User
 ├── name
 ├── email
 └── address
      ├── city
      └── pincode
```

---

7. Optional Fields

An optional field does not have to be provided.

Pydantic v2 example:

```
from typing import Optional

class User(BaseModel):
    name: str
    email: Optional[str] = None
```

Valid:

```
{
  "name": "John"
}
```

Also valid:

```
{
  "name": "John",
  "email": "john@gmail.com"
}
```

Important

email: Optional[str] = None

means:

email can contain a string or None, and it is not required.

---

8. Default Values

A field can have a default value.

```
class User(BaseModel):
    name: str
    country: str = "India"
```

If the client sends:

```
{
  "name": "John"
}
```

Pydantic automatically uses:

country = India

---

9. Request Body Validation

FastAPI automatically validates request data using Pydantic.

Example:

```
class User(BaseModel):
    name: str
    age: int
```

Correct:

```
{
  "name": "John",
  "age": 25
}
```

Incorrect:

```
{
  "name": "John",
  "age": "hello"
}
```

FastAPI returns a validation error.

Common validations

- Data type
- Required fields
- String length
- Numeric limits
- Email format
- Custom rules

Interview Point

FastAPI uses Pydantic to validate request data before passing it to the path operation function.

---

10. Request Body Documentation

FastAPI automatically generates API documentation.

You can open:

http://127.0.0.1:8000/docs

It shows:

- Request body schema
- Required fields
- Data types
- Validation rules
- Example request

FastAPI generates this documentation using **OpenAPI**.

Module 7 — Pydantic

1. What is Pydantic?

Pydantic is a Python library used for data validation and data parsing using Python type hints.

FastAPI heavily uses Pydantic for:

- Request validation
- Response validation
- Data parsing
- Serialization
- Schema generation

Interview Definition

Pydantic is a Python data validation library that uses type hints to validate and parse data.

---

2. Why FastAPI Uses Pydantic?

Pydantic makes it easy to:

- Define data structure
- Validate input
- Convert compatible input types
- Handle nested data
- Generate schemas
- Create API documentation

Example:

```
class User(BaseModel):
    name: str
    age: int
```

FastAPI knows:

name → string
age → integer

---

3. BaseModel

BaseModel is the main class used to create Pydantic models.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
```

User is now a Pydantic model.

---

4. Fields

Fields are the attributes defined inside a Pydantic model.

```
class User(BaseModel):
    name: str
    age: int
    email: str
```

Here:

name
age
email

are fields.

---

5. Type Hints

Pydantic uses Python type hints to understand expected data types.

```
class User(BaseModel):
    name: str
    age: int
    salary: float
    active: bool
```

Types:

name → str
age → int
salary → float
active → bool

---

6. Required Fields

A field without a default value is generally required.

```
class User(BaseModel):
```

```
    name: str
    age: int
```

Both fields are required.

This is invalid:

```
{
  "name": "John"
}
```

because age is missing.

---

7. Optional Fields

Optional fields can be omitted when given an appropriate default.

```
class User(BaseModel):
    name: str
    phone: str | None = None
```

phone is optional.

---

8. Default Values

```
class User(BaseModel):
    name: str
    country: str = "India"
```

If country is not provided:

country = India

---

9. Nested Models

One Pydantic model can contain another.

```
class Address(BaseModel):
    city: str
    pincode: int
```

```
class User(BaseModel):
    name: str
    address: Address
```

This is useful for complex JSON structures.

---

10. Lists

Pydantic supports lists with type information.

```
class Student(BaseModel):
    name: str
    subjects: list[str]
```

Example:

```
{
  "name": "John",
  "subjects": ["Python", "SQL", "FastAPI"]
}
```

You can also use:

```
list[int]
```

---

11. Dictionaries

Pydantic can validate dictionary keys and values.

```
class Student(BaseModel):
    marks: dict[str, int]
```

Example:

```
{
  "marks": {
    "Python": 90,
    "SQL": 85
  }
}
```

Meaning:

key → string
value → integer

---

12. Enums

Enums are used when a field should accept only predefined values.

```
from enum import Enum

class Role(str, Enum):
    admin = "admin"
    user = "user"
```

Then:

```
class User(BaseModel):
    name: str
    role: Role
```

Valid:

```
{
  "name": "John",
  "role": "admin"
}
```

Invalid values such as "manager" will fail validation.

Interview Point

Enum restricts a field to a predefined set of values.

---

13. Field Validation

Pydantic allows you to define rules for fields.

For example:

age must be >= 18
username must have minimum length
price must be greater than 0

---

14. Field()

Field() allows you to add constraints and metadata to fields.

Example:

```
from pydantic import BaseModel, Field

class User(BaseModel):
    name: str = Field(min_length=3)
    age: int = Field(ge=18)
```

Meaning:

name → minimum 3 characters
age → greater than or equal to 18

Common constraints:

min_length
max_length
gt
ge
lt
le

Where:

gt = greater than
ge = greater than or equal
lt = less than
le = less than or equal

---

15. Custom Validation

Sometimes built-in validation is not enough.

Pydantic allows custom validation logic.

In Pydantic v2, commonly use:

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    username: str
```

```
    @field_validator("username")
    @classmethod
    def validate_username(cls, value):
        if " " in value:
            raise ValueError("Username cannot contain spaces")
        return value
```

Now:

john123 → valid
john 123 → invalid

Interview Point

Custom validators are used when business-specific validation rules are required.

---

16. Model Serialization

Serialization means converting a Python/Pydantic object into a format that can be sent or stored.

For example:

Pydantic Model
↓
Dictionary / JSON

In Pydantic v2:

user.model_dump()

returns a Python dictionary.

---

17. Model Deserialization

Deserialization means converting external data into a Python/Pydantic model.

For example:

JSON / Dictionary
↓
Pydantic Model

Pydantic validates the incoming data during this process.

---

18. Pydantic v2 Concepts

Pydantic v2 introduced several updated APIs.

Important methods to remember:

model_dump()
model_validate()

Older Pydantic v1 methods such as:

.dict()
.parse_obj()

have modern v2 equivalents.

Important Interview Point

If using modern FastAPI projects, know the Pydantic v2 APIs.

---

19. model_dump()

Used to convert a Pydantic model into a Python dictionary.

```
user = User(name="John", age=25)

data = user.model_dump()

print(data)
```

Output:

```
{
   "name": "John",
   "age": 25
}
```

Think:

Model → Dictionary

---

20. model_validate()

Used to create/validate a Pydantic model from data such as a dictionary.

```
data = {
   "name": "John",
   "age": 25
}

user = User.model_validate(data)
```

Think:

Dictionary → Model

Module 8 — Response Models

1. What is a Response Model?

A **response model** defines the structure of data that an API should return to the client.

Example:

```
class UserResponse(BaseModel):
   name: str
   age: int
```

The API should return data matching this structure.

Interview Definition

A response model defines and validates the structure of the data returned by an API.

---

2. response_model

FastAPI uses the response_model parameter to specify the expected response structure.

```
@app.get("/users", response_model=UserResponse)
def get_user():
   return {
      "name": "John",
      "age": 25
   }
```

FastAPI uses UserResponse to:

- Validate response data
- Serialize response
- Generate documentation
- Filter unwanted fields

---

3. Response Validation

FastAPI validates the returned data against the response model.

Example:

```
class UserResponse(BaseModel):
    name: str
    age: int
```

If your endpoint returns incorrect data:

```
return {
    "name": "John",
    "age": "abc"
}
```

the response does not match the declared schema.

Interview Point

response_model provides output validation and serialization.

---

4. Response Serialization

Response data needs to be converted into a format that can be sent over HTTP, commonly JSON.

FastAPI handles this serialization automatically.

Conceptually:

```
Python Object
    ↓
Pydantic/FastAPI
    ↓
```

JSON Response

---

5. Returning Pydantic Models

You can directly return a Pydantic model.

```
class UserResponse(BaseModel):
    name: str
    age: int

@app.get("/user", response_model=UserResponse)
def get_user():
    return UserResponse(
        name="John",
        age=25
    )
```

FastAPI converts the response into JSON.

---

6. Filtering Response Fields

One major benefit of response_model is that it can prevent unwanted fields from being returned.

Suppose database data contains:

```
{
    "name": "John",
    "email": "john@gmail.com",
    "password": "secret123"
}
```

But response model is:

```
class UserResponse(BaseModel):
    name: str
    email: str
```

The response contains:

```
{
  "name": "John",
  "email": "john@gmail.com"
}
```

The password is excluded because it is not part of the response model.

Very Important Interview Point

Response models help control what data is exposed to API clients.

---

7. Input Model vs Output Model

It is a good practice to use **different models for input and output**.

Input model

```
class UserCreate(BaseModel):
    name: str
    email: str
    password: str
```

Output model

```
class UserResponse(BaseModel):
    id: int
    name: str
    email: str
```

Notice:

Input:
name
email
password

Output:
id
name
email

We don't return the password.

Interview Question

Why use separate request and response models?

Answer:

To clearly separate input data from output data and prevent sensitive or unwanted fields from being exposed.

---

8. Multiple Response Types

Sometimes an endpoint can return different response structures depending on the situation.

For example:

200 → User data
404 → User not found

FastAPI allows different response definitions using response configuration and status codes.

For simple interview understanding:

Success Response → UserResponse
Error Response → ErrorResponse

The important idea is that APIs may have different response schemas for different outcomes.

---

9. response_model_exclude

Used to exclude specific fields from the response.

Example:

```
@app.get(
    "/user",
    response_model=UserResponse,
    response_model_exclude={"email"}
)
def get_user():
    ...
```

The email field will be excluded from the response.

---

10. response_model_include

Used to include only specific fields.

Example:

```
@app.get(
    "/user",
    response_model=UserResponse,
    response_model_include={"name"}
)
def get_user():
    ...
```

Only the name field is included.

Note

For larger applications, **separate Pydantic response models are usually clearer and easier to maintain** than heavily relying on include/exclude options.

Module 10 — Error Handling

1. What is Exception Handling?

Exception handling means **handling errors that occur while the application is running**.

Without proper handling:

```
Request
  ↓
Error
  ↓
Application may fail
```

With exception handling:

```
Request
  ↓
Error
  ↓
Handle Error
  ↓
Proper HTTP Response
```

Python commonly uses:

```
try:
    ...
except:
```

```
    ...
```

FastAPI also provides HTTP-specific exception handling.

---

2. HTTPException

HTTPException is used to return an HTTP error response from a FastAPI endpoint.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id != 1:
        raise HTTPException(
            status_code=404,
            detail="User not found"
        )

    return {"id": 1, "name": "John"}
```

If the user doesn't exist:

```
{
    "detail": "User not found"
}
```

with status:

404 Not Found

Common status codes

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
500	Internal Server Error

---

3. Raising HTTP Exceptions

In FastAPI, we use:

raise HTTPException(...)

Example:

```
if not user:
    raise HTTPException(
        status_code=404,
        detail="User not found"
    )
```

Important

Use raise, not return.

```
# Correct
raise HTTPException(status_code=404, detail="Not found")
```

---

4. Custom Exceptions

A custom exception is an exception class created for your application's specific business logic.

Example:

```
class UserNotFoundException(Exception):
    pass
```

Then:

```
if user is None:
    raise UserNotFoundException()
```

This is useful when you want to separate **business errors** from normal HTTP errors.

---

5. Custom Exception Handlers

A custom exception handler tells FastAPI:

"When this exception occurs, return this particular response."

Example:

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

class UserNotFoundException(Exception):
    pass

@app.exception_handler(UserNotFoundException)
async def user_not_found_handler(
    request: Request,
    exc: UserNotFoundException
):
    return JSONResponse(
        status_code=404,
        content={"detail": "User not found"}
    )
```

Now whenever:

```
raise UserNotFoundException()
```

occurs, FastAPI uses the custom handler.

---

6. Validation Errors

FastAPI automatically validates:

- Path parameters
- Query parameters
- Request bodies
- Types
- Pydantic models

Example:

```
class User(BaseModel):
    name: str
    age: int
```

If the client sends:

```
{
   "name": "John",
   "age": "hello"
}
```

validation fails.

FastAPI automatically returns a validation error response.

---

7. RequestValidationError

RequestValidationError represents validation errors that occur while processing an incoming request.

You can create a custom handler for it.

from fastapi.exceptions import RequestValidationError

Example:

```
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(
   request: Request,
   exc: RequestValidationError
):
   return JSONResponse(
      status_code=422,
      content={
         "detail": "Invalid request data"
      }
   )
```

Interview Point

RequestValidationError is used to handle validation errors generated from invalid incoming request data.

---

8. Global Exception Handlers

A global exception handler handles a particular type of exception across the entire application.

Example:

```
@app.exception_handler(Exception)
async def global_exception_handler(
   request: Request,
   exc: Exception
):
   return JSONResponse(
      status_code=500,
      content={"detail": "Internal server error"}
   )
```

This prevents exposing internal implementation details to users.

Important

In production, don't return sensitive exception information such as:

str(exc)

directly to clients.

Instead, log the actual error internally and return a safe message.

---

9. Error Response Format

A good API should return a **consistent error format**.

Example:

```
{
   "detail": "User not found"
}
```

A custom application might use:

```
{
   "status": 404,
   "message": "User not found",
   "error": "USER_NOT_FOUND"
}
```

Consistency makes frontend development easier.

Module 11 — Dependency Injection

Very important FastAPI interview topic.

1. What is Dependency Injection?

Dependency Injection means:

A function receives the resources or services it needs from an external source instead of creating them itself.

Simple example:

```
def get_database():
    return database_connection
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_database)):
    ...
```

FastAPI automatically calls get_database() and provides its result to db.

---

2. Why Dependency Injection?

Dependency Injection helps with:

- Code reuse
- Separation of concerns
- Authentication
- Database connections
- Common parameters
- Testing
- Cleaner code

Without dependency injection:

```
@app.get("/users")
def get_users():
    db = create_database_connection()
```

With dependency injection:

```
@app.get("/users")
def get_users(db = Depends(get_database)):
    ...
```

The endpoint doesn't need to know how the database connection is created.

---

3. Depends()

Depends() tells FastAPI:

"This parameter should be provided by another dependency function."

Example:

```
from fastapi import Depends

def get_user():
    return {"name": "John"}

@app.get("/profile")
def profile(user = Depends(get_user)):
    return user
```

Flow:

```
Request
  ↓
Depends(get_user)
  ↓
get_user()
  ↓
user
  ↓
profile()
```

---

4. Dependency Functions

A dependency is usually a normal Python function.

```
def get_current_user():
    return {
```

```
    "id": 1,
    "name": "John"
}
```

Use it:

```
@app.get("/profile")
def profile(user = Depends(get_current_user)):
    return user
```

---

5. Dependency with Parameters

Dependencies can themselves have parameters.

```
def get_user(user_id: int):
    return {
        "id": user_id
    }
```

Then:

```
@app.get("/users/{user_id}")
def get_user_data(
    user = Depends(get_user)
):
    return user
```

FastAPI resolves the dependency's parameters from the request when applicable.

---

6. Multiple Dependencies

An endpoint can have multiple dependencies.

```
@app.get("/admin")
def admin_page(
    user = Depends(get_current_user),
    db = Depends(get_database)
):
    return {"message": "Admin page"}
```

Here there are two dependencies:

get_current_user()

get_database()

---

7. Nested Dependencies

A dependency can depend on another dependency.

```
def get_database():
    return database


def get_current_user(
    db = Depends(get_database)
):
    return get_user_from_db(db)
```

Then:

```
@app.get("/profile")
def profile(
    user = Depends(get_current_user)
):
    return user
```

Flow:

```
Endpoint
  ↓
get_current_user
  ↓
get_database
```

This is called **nested dependency injection**.

---

8. Reusable Dependencies

The biggest advantage is that dependencies can be reused across multiple endpoints.

```
def get_current_user():
    ...
```

Use it in:

```
@app.get("/profile")
```

```
def profile(user = Depends(get_current_user)):
    ...

@app.get("/orders")
def orders(user = Depends(get_current_user)):
    ...

@app.get("/dashboard")
def dashboard(user = Depends(get_current_user)):
    ...
```

Write once, reuse many times.

---

9. Database Dependency

A common FastAPI pattern is creating a database session dependency.

Conceptually:

```
def get_db():
    db = create_database_session()

    try:
        yield db
    finally:
        db.close()
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
    return db.query(...)
```

Flow:

```
Request
  ↓
get_db()
  ↓
Database Session
  ↓
Endpoint
  ↓
Response
```

↓
Session Cleanup

Interview Point

Database dependencies are commonly used to create, provide, and clean up database sessions.

---

10. Authentication Dependency

Authentication is another major use case.

```
def get_current_user():
    # verify token
    # identify user
    return user
```

Then:

```
@app.get("/profile")
def profile(
    user = Depends(get_current_user)
):
    return user
```

Every protected endpoint can reuse the same authentication dependency.

---

11. Dependency Overrides

Dependency overrides are mainly useful for **testing**.

Suppose production uses:

```
def get_database():
    return real_database
```

During testing, you can replace it:

```
app.dependency_overrides[get_database] = get_test_database
```

Now FastAPI uses the test dependency instead.

Interview Point

Dependency overrides allow us to replace dependencies, especially during testing.

---

12. Dependency Lifecycle

A dependency can have setup and cleanup logic.

The common pattern uses yield:

```
def get_db():
    db = create_session()

    try:
        yield db
    finally:
        db.close()
```

Think:

```
Setup
  ↓
yield resource
  ↓
Endpoint uses resource
  ↓
Cleanup
```

Module 12 — Routers

1. Why Use Routers?

As an application grows, putting every endpoint in main.py becomes difficult to maintain.

Instead of:

```
main.py
 ├── users
 ├── products
 ├── orders
 ├── payments
 ├── authentication
 └── ...
```

we separate routes:

main.py

```
routers/
 ├── users.py
 ├── products.py
 └── orders.py
```

Main Benefit

Routers help organize large FastAPI applications into smaller, maintainable modules.

---

2. APIRouter

APIRouter is used to create groups of related API routes.

Example:

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/users")
def get_users():
    return {"users": []}
```

---

3. Creating Separate Route Files

Example:

```
app/
 ├── main.py
 └── routers/
     ├── users.py
     ├── products.py
     └── orders.py
```

users.py:

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/")
def get_users():
```

```
    return {"message": "Users"}
```

This keeps user-related endpoints together.

---

4. include_router()

include_router() connects a router to the main FastAPI application.

main.py:

```
from fastapi import FastAPI
from routers import users

app = FastAPI()

app.include_router(users.router)
```

Now routes defined inside users.py become part of the application.

---

5. Router Prefixes

A prefix adds a common path to all routes in a router.

```
app.include_router(
    users.router,
    prefix="/users"
)
```

If router contains:

```
@router.get("/")
```

Final endpoint becomes:

GET /users/

Another example:

```
app.include_router(
    products.router,
    prefix="/products"
)
```

---

6. Router Tags

Tags group endpoints in Swagger documentation.

```
app.include_router(
    users.router,
    prefix="/users",
    tags=["Users"]
)
```

In /docs, these endpoints appear under:

Users

---

7. Router Dependencies

Dependencies can be applied to an entire router.

```
app.include_router(
    admin.router,
    prefix="/admin",
    dependencies=[Depends(get_current_user)]
)
```

Now the dependency applies to routes in that router.

Useful for:

- Authentication
- Authorization
- Common checks

---

8. Organizing Large Applications

A common project structure:

```
app/
├── main.py
├── routers/
│   ├── users.py
│   ├── products.py
```

```
    └── orders.py
├── models/
├── schemas/
├── services/
└── database/
```

Typical responsibility:

Folder	Purpose
main.py	Application entry point
routers/	API endpoints
models/	Database models
schemas/	Pydantic schemas
services/	Business logic
database/	DB configuration/session

Interview Point

Routers separate API routes by feature and improve scalability and maintainability.

Module 13 — Middleware

1. What is Middleware?

Middleware is code that runs **between the incoming request and the outgoing response**.

Conceptually:

```
Client
  ↓
Request
  ↓
Middleware
  ↓
FastAPI Endpoint
  ↓
Middleware
  ↓
Response
  ↓
Client
```

Middleware can inspect or modify requests and responses.

---

2. Request Middleware

Request middleware runs before the request reaches the endpoint.

Example use cases:

- Logging
- Authentication checks
- Request timing
- Adding request information

---

3. Response Middleware

After the endpoint produces a response, middleware can process the response before it goes back to the client.

Conceptually:

Request
↓
Middleware
↓
Endpoint
↓
Response
↓
Middleware
↓
Client

---

4. @app.middleware("http")

FastAPI allows custom HTTP middleware using:

```
@app.middleware("http")
async def my_middleware(request, call_next):
    response = await call_next(request)
    return response
```

call_next(request) passes the request to the next middleware or endpoint.

---

5. Middleware Execution Flow

Think of middleware like layers:

Client
↓
Middleware A
↓
Middleware B
↓
Endpoint
↑
Middleware B
↑
Middleware A
↑
Client

Request goes **down** through middleware.

Response comes **back up** through middleware.

---

6. Custom Middleware

Example:

```
@app.middleware("http")
async def custom_middleware(request, call_next):

    print("Request received")

    response = await call_next(request)

    print("Response generated")

    return response
```

---

7. Logging Middleware

Middleware can log:

HTTP method
URL
Status code
Processing time

Example:

```
@app.middleware("http")
async def logging_middleware(request, call_next):

    print(request.method, request.url)

    response = await call_next(request)

    print(response.status_code)

    return response
```

---

8. Timing Middleware

Used to measure API execution time.

Conceptually:

```
start = time.time()

response = await call_next(request)

duration = time.time() - start
```

Then log:

GET /users → 0.025 seconds

Useful for performance monitoring.

---

9. Authentication Middleware

Middleware can inspect requests and perform common authentication-related processing.

However, in FastAPI, **dependencies are often a better fit for endpoint-specific authentication/authorization**, because dependencies integrate naturally with route handling and can provide the authenticated user to the endpoint.

Interview Point

Middleware is suitable for cross-cutting concerns, while dependencies are often better for route-specific authentication and authorization.

---

10. CORS Middleware

FastAPI provides:

CORSMiddleware

Example:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This allows a frontend running on the specified origin to communicate with the FastAPI backend, subject to the configured CORS rules.

Module 14 — CORS

Very important for real-world FastAPI projects.

1. What is CORS?

CORS stands for:

Cross-Origin Resource Sharing

It is a browser security mechanism that controls whether a web page from one **origin** can make requests to a server at another origin.

Example:

Frontend:
http://localhost:3000

Backend:
http://localhost:8000

These are different origins because their ports differ.

The browser applies CORS rules when the frontend tries to call the backend.

---

2. Why CORS Occurs?

Suppose:

React Frontend
localhost:3000
↓
FastAPI Backend
localhost:8000

The browser sees different origins.

Without appropriate CORS configuration, the browser may block the frontend from accessing the backend response.

Important

CORS is primarily a **browser security mechanism**.

It is not simply a FastAPI error.

---

3. Browser Security

Browsers restrict cross-origin requests to protect users from malicious websites accessing resources they should not access.

The server can tell the browser which origins are allowed through CORS response headers.

---

4. CORSMiddleware

FastAPI uses Starlette's CORSMiddleware.

from fastapi.middleware.cors import CORSMiddleware

Example:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

---

5. Allowed Origins

An **origin** consists of:

scheme + host + port

Example:

http://localhost:3000

You can specify allowed frontend origins:

```
allow_origins=[
    "http://localhost:3000",
    "https://myfrontend.com"
]
```

Development

You may temporarily use:

```
allow_origins=["*"]
```

But in production, it's generally better to specify trusted origins.

---

6. Allowed Methods

You can specify which HTTP methods are allowed:

```
allow_methods=[
   "GET",
   "POST",
   "PUT",
   "DELETE"
]
```

Or:

```
allow_methods=["*"]
```

Common methods:

GET
POST
PUT
PATCH
DELETE

---

7. Allowed Headers

Headers sent by the frontend can also be controlled.

```
allow_headers=[
   "Content-Type",
   "Authorization"
]
```

Or:

```
allow_headers=["*"]
```

Common examples:

Content-Type
Authorization
Accept

---

8. Credentials

Credentials can include things such as:

- Cookies
- Browser authentication information

- Other credentialed cross-origin requests

FastAPI:

allow_credentials=True

Example:

```
app.add_middleware(
   CORSMiddleware,
   allow_origins=["http://localhost:3000"],
   allow_credentials=True,
   allow_methods=["*"],
   allow_headers=["*"],
)
```

Important

When credentials are enabled, you should configure trusted origins appropriately rather than relying on an unrestricted wildcard origin.

---

9. Frontend + FastAPI Communication

Typical architecture:

```
React
localhost:3000
     |
     | HTTP Request
     ↓
FastAPI
localhost:8000
     |
     ↓
Database
```

If frontend and backend have different origins:

```
Browser
  ↓
CORS Check
  ↓
FastAPI
```

FastAPI's CORS middleware sends the appropriate headers so the browser knows whether the frontend is allowed to access the response.

Module 6 — Request Body

1. What is Request Body?

The **request body** contains data sent by the client to the server.

It is mainly used with:

- POST
- PUT
- PATCH

Example:

```
{
  "name": "Prathamesh",
  "age": 22
}
```

Here, the JSON data is the **request body**.

Interview Point

Request body is data sent from the client to the server, usually in JSON format.

---

2. JSON Request Body

JSON is the most common format for sending request body data in REST APIs.

Example:

```
{
  "name": "John",
  "email": "john@gmail.com",
  "age": 25
}
```

FastAPI can automatically parse JSON data received from the client.

---

3. Request Body Using Pydantic

FastAPI commonly uses **Pydantic models** to define the structure of request data.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return user
```

The client sends:

```
{
  "name": "John",
  "age": 25
}
```

FastAPI:

1. Receives JSON
2. Converts it into a Python object
3. Validates the data
4. Passes it to the function

Interview Point

Pydantic models define and validate the expected structure of request bodies.

---

4. Single Body Parameter

A single Pydantic model can represent the complete request body.

```
class Product(BaseModel):
    name: str
    price: float

@app.post("/products")
```

```
def create_product(product: Product):
    return product
```

Request:

```
{
  "name": "Laptop",
  "price": 50000
}
```

Here, product is the single body parameter.

---

5. Multiple Body Parameters

FastAPI can accept multiple Pydantic models in one request.

```
class User(BaseModel):
    name: str

class Address(BaseModel):
    city: str

@app.post("/users")
def create_user(user: User, address: Address):
    return {
        "user": user,
        "address": address
    }
```

Request:

```
{
  "user": {
    "name": "John"
  },
  "address": {
    "city": "Pune"
  }
}
```

Interview Point

Multiple body parameters are represented as separate fields in the JSON request.

---

6. Nested Request Bodies

A Pydantic model can contain another Pydantic model.

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    address: Address
```

Request:

```
{
  "name": "John",
  "address": {
    "city": "Pune",
    "pincode": 411001
  }
}
```

This is called a **nested request body**.

Real-world Example

```
User
 ├── name
 ├── email
 └── address
      ├── city
      └── pincode
```

---

7. Optional Fields

An optional field does not have to be provided.

Pydantic v2 example:

```
from typing import Optional

class User(BaseModel):
    name: str
```

```
    email: Optional[str] = None
```

Valid:

```
{
  "name": "John"
}
```

Also valid:

```
{
  "name": "John",
  "email": "john@gmail.com"
}
```

Important

email: Optional[str] = None

means:

email can contain a string or None, and it is not required.

---

8. Default Values

A field can have a default value.

```
class User(BaseModel):
    name: str
    country: str = "India"
```

If the client sends:

```
{
  "name": "John"
}
```

Pydantic automatically uses:

country = India

---

9. Request Body Validation

FastAPI automatically validates request data using Pydantic.

Example:

```
class User(BaseModel):
    name: str
    age: int
```

Correct:

```
{
  "name": "John",
  "age": 25
}
```

Incorrect:

```
{
  "name": "John",
  "age": "hello"
}
```

FastAPI returns a validation error.

Common validations

- Data type
- Required fields
- String length
- Numeric limits
- Email format
- Custom rules

Interview Point

FastAPI uses Pydantic to validate request data before passing it to the path operation function.

---

10. Request Body Documentation

FastAPI automatically generates API documentation.

You can open:

http://127.0.0.1:8000/docs

It shows:

- Request body schema
- Required fields
- Data types
- Validation rules
- Example request

FastAPI generates this documentation using **OpenAPI**.

Module 7 — Pydantic

1. What is Pydantic?

Pydantic is a Python library used for data validation and data parsing using Python type hints.

FastAPI heavily uses Pydantic for:

- Request validation
- Response validation
- Data parsing
- Serialization
- Schema generation

Interview Definition

Pydantic is a Python data validation library that uses type hints to validate and parse data.

---

2. Why FastAPI Uses Pydantic?

Pydantic makes it easy to:

- Define data structure
- Validate input
- Convert compatible input types
- Handle nested data
- Generate schemas
- Create API documentation

Example:

```
class User(BaseModel):
    name: str
    age: int
```

FastAPI knows:

name → string
age → integer

---

3. BaseModel

BaseModel is the main class used to create Pydantic models.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
```

User is now a Pydantic model.

---

4. Fields

Fields are the attributes defined inside a Pydantic model.

```
class User(BaseModel):
    name: str
    age: int
    email: str
```

Here:

name
age
email

are fields.

---

5. Type Hints

Pydantic uses Python type hints to understand expected data types.

```
class User(BaseModel):
    name: str
    age: int
    salary: float
    active: bool
```

Types:

```
name   → str
age    → int
salary → float
active → bool
```

---

6. Required Fields

A field without a default value is generally required.

```
class User(BaseModel):
    name: str
    age: int
```

Both fields are required.

This is invalid:

```
{
  "name": "John"
}
```

because age is missing.

---

7. Optional Fields

Optional fields can be omitted when given an appropriate default.

```
class User(BaseModel):
    name: str
    phone: str | None = None
```

phone is optional.

---

8. Default Values

```
class User(BaseModel):
    name: str
    country: str = "India"
```

If country is not provided:

country = India

---

9. Nested Models

One Pydantic model can contain another.

```
class Address(BaseModel):
    city: str
    pincode: int

class User(BaseModel):
    name: str
    address: Address
```

This is useful for complex JSON structures.

---

10. Lists

Pydantic supports lists with type information.

```
class Student(BaseModel):
    name: str
    subjects: list[str]
```

Example:

```
{
  "name": "John",
  "subjects": ["Python", "SQL", "FastAPI"]
}
```

You can also use:

list[int]

---

11. Dictionaries

Pydantic can validate dictionary keys and values.

```
class Student(BaseModel):
    marks: dict[str, int]
```

Example:

```
{
  "marks": {
    "Python": 90,
    "SQL": 85
  }
}
```

Meaning:

key → string
value → integer

---

12. Enums

Enums are used when a field should accept only predefined values.

```
from enum import Enum

class Role(str, Enum):
    admin = "admin"
    user = "user"
```

Then:

```
class User(BaseModel):
    name: str
    role: Role
```

Valid:

```
{
  "name": "John",
  "role": "admin"
}
```

Invalid values such as "manager" will fail validation.

Interview Point

Enum restricts a field to a predefined set of values.

---

13. Field Validation

Pydantic allows you to define rules for fields.

For example:

age must be >= 18
username must have minimum length
price must be greater than 0

---

14. Field()

Field() allows you to add constraints and metadata to fields.

Example:

```
from pydantic import BaseModel, Field

class User(BaseModel):
    name: str = Field(min_length=3)
    age: int = Field(ge=18)
```

Meaning:

name → minimum 3 characters
age → greater than or equal to 18

Common constraints:

min_length
max_length
gt
ge
lt
le

Where:

gt = greater than

ge = greater than or equal
lt = less than
le = less than or equal

---

15. Custom Validation

Sometimes built-in validation is not enough.

Pydantic allows custom validation logic.

In Pydantic v2, commonly use:

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    username: str

    @field_validator("username")
    @classmethod
    def validate_username(cls, value):
        if " " in value:
            raise ValueError("Username cannot contain spaces")
        return value
```

Now:

john123 → valid
john 123 → invalid

Interview Point

Custom validators are used when business-specific validation rules are required.

---

16. Model Serialization

Serialization means converting a Python/Pydantic object into a format that can be sent or stored.

For example:

Pydantic Model
↓
Dictionary / JSON

In Pydantic v2:

user.model_dump()

returns a Python dictionary.

---

17. Model Deserialization

Deserialization means converting external data into a Python/Pydantic model.

For example:

JSON / Dictionary
↓
Pydantic Model

Pydantic validates the incoming data during this process.

---

18. Pydantic v2 Concepts

Pydantic v2 introduced several updated APIs.

Important methods to remember:

model_dump()
model_validate()

Older Pydantic v1 methods such as:

.dict()
.parse_obj()

have modern v2 equivalents.

Important Interview Point

If using modern FastAPI projects, know the Pydantic v2 APIs.

---

19. model_dump()

Used to convert a Pydantic model into a Python dictionary.

```
user = User(name="John", age=25)

data = user.model_dump()

print(data)
```

Output:

```
{
   "name": "John",
   "age": 25
}
```

Think:

Model → Dictionary

---

20. model_validate()

Used to create/validate a Pydantic model from data such as a dictionary.

```
data = {
   "name": "John",
   "age": 25
}

user = User.model_validate(data)
```

Think:

Dictionary → Model

Module 8 — Response Models

1. What is a Response Model?

A **response model** defines the structure of data that an API should return to the client.

Example:

```
class UserResponse(BaseModel):
   name: str
```

```
    age: int
```

The API should return data matching this structure.

Interview Definition

A response model defines and validates the structure of the data returned by an API.

---

2. response_model

FastAPI uses the response_model parameter to specify the expected response structure.

```
@app.get("/users", response_model=UserResponse)
def get_user():
    return {
        "name": "John",
        "age": 25
    }
```

FastAPI uses UserResponse to:

- Validate response data
- Serialize response
- Generate documentation
- Filter unwanted fields

---

3. Response Validation

FastAPI validates the returned data against the response model.

Example:

```
class UserResponse(BaseModel):
    name: str
    age: int
```

If your endpoint returns incorrect data:

```
return {
    "name": "John",
    "age": "abc"
}
```

the response does not match the declared schema.

Interview Point

response_model provides output validation and serialization.

---

4. Response Serialization

Response data needs to be converted into a format that can be sent over HTTP, commonly JSON.

FastAPI handles this serialization automatically.

Conceptually:

Python Object
↓
Pydantic/FastAPI
↓
JSON Response

---

5. Returning Pydantic Models

You can directly return a Pydantic model.

```
class UserResponse(BaseModel):
    name: str
    age: int

@app.get("/user", response_model=UserResponse)
def get_user():
    return UserResponse(
        name="John",
        age=25
    )
```

FastAPI converts the response into JSON.

---

6. Filtering Response Fields

One major benefit of response_model is that it can prevent unwanted fields from being returned.

Suppose database data contains:

```
{
   "name": "John",
   "email": "john@gmail.com",
   "password": "secret123"
}
```

But response model is:

```
class UserResponse(BaseModel):
   name: str
   email: str
```

The response contains:

```
{
  "name": "John",
  "email": "john@gmail.com"
}
```

The password is excluded because it is not part of the response model.

Very Important Interview Point

Response models help control what data is exposed to API clients.

---

7. Input Model vs Output Model

It is a good practice to use **different models for input and output**.

Input model

```
class UserCreate(BaseModel):
   name: str
   email: str
   password: str
```

Output model

```
class UserResponse(BaseModel):
   id: int
   name: str
   email: str
```

Notice:

Input:
name
email
password

Output:
id
name
email

We don't return the password.

Interview Question

Why use separate request and response models?

Answer:

To clearly separate input data from output data and prevent sensitive or unwanted fields from being exposed.

---

8. Multiple Response Types

Sometimes an endpoint can return different response structures depending on the situation.

For example:

200 → User data
404 → User not found

FastAPI allows different response definitions using response configuration and status codes.

For simple interview understanding:

Success Response → UserResponse
Error Response → ErrorResponse

The important idea is that APIs may have different response schemas for different outcomes.

---

9. response_model_exclude

Used to exclude specific fields from the response.

Example:

```
@app.get(
    "/user",
    response_model=UserResponse,
    response_model_exclude={"email"}
)
def get_user():
    ...
```

The email field will be excluded from the response.

---

10. response_model_include

Used to include only specific fields.

Example:

```
@app.get(
    "/user",
    response_model=UserResponse,
    response_model_include={"name"}
)
def get_user():
    ...
```

Only the name field is included.

Note

For larger applications, **separate Pydantic response models are usually clearer and easier to maintain** than heavily relying on include/exclude options.

Module 10 — Error Handling

1. What is Exception Handling?

Exception handling means **handling errors that occur while the application is running**.

Without proper handling:

Request
↓
Error
↓
Application may fail

With exception handling:

Request
↓
Error
↓
Handle Error
↓
Proper HTTP Response

Python commonly uses:

```
try:
    ...
except:
    ...
```

FastAPI also provides HTTP-specific exception handling.

---

2. HTTPException

HTTPException is used to return an HTTP error response from a FastAPI endpoint.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id != 1:
        raise HTTPException(
            status_code=404,
            detail="User not found"
        )

    return {"id": 1, "name": "John"}
```

If the user doesn't exist:

```
{
   "detail": "User not found"
}
```

with status:

404 Not Found

Common status codes

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
500	Internal Server Error

---

3. Raising HTTP Exceptions

In FastAPI, we use:

raise HTTPException(...)

Example:

```
if not user:
   raise HTTPException(
      status_code=404,
      detail="User not found"
   )
```

Important

Use raise, not return.

```
# Correct
raise HTTPException(status_code=404, detail="Not found")
```

---

4. Custom Exceptions

A custom exception is an exception class created for your application's specific business logic.

Example:

```
class UserNotFoundException(Exception):
    pass
```

Then:

```
if user is None:
    raise UserNotFoundException()
```

This is useful when you want to separate **business errors** from normal HTTP errors.

---

5. Custom Exception Handlers

A custom exception handler tells FastAPI:

"When this exception occurs, return this particular response."

Example:

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

class UserNotFoundException(Exception):
    pass

@app.exception_handler(UserNotFoundException)
async def user_not_found_handler(
    request: Request,
    exc: UserNotFoundException
):
    return JSONResponse(
        status_code=404,
        content={"detail": "User not found"}
    )
```

Now whenever:

```
raise UserNotFoundException()
```

occurs, FastAPI uses the custom handler.

---

6. Validation Errors

FastAPI automatically validates:

- Path parameters
- Query parameters
- Request bodies
- Types
- Pydantic models

Example:

```
class User(BaseModel):
   name: str
   age: int
```

If the client sends:

```
{
   "name": "John",
   "age": "hello"
}
```

validation fails.

FastAPI automatically returns a validation error response.

---

7. RequestValidationError

RequestValidationError represents validation errors that occur while processing an incoming request.

You can create a custom handler for it.

from fastapi.exceptions import RequestValidationError

Example:

```
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(
```

```
    request: Request,
    exc: RequestValidationError
):
    return JSONResponse(
        status_code=422,
        content={
            "detail": "Invalid request data"
        }
    )
```

Interview Point

RequestValidationError is used to handle validation errors generated from invalid incoming request data.

---

8. Global Exception Handlers

A global exception handler handles a particular type of exception across the entire application.

Example:

```
@app.exception_handler(Exception)
async def global_exception_handler(
    request: Request,
    exc: Exception
):
    return JSONResponse(
        status_code=500,
        content={"detail": "Internal server error"}
    )
```

This prevents exposing internal implementation details to users.

Important

In production, don't return sensitive exception information such as:

str(exc)

directly to clients.

Instead, log the actual error internally and return a safe message.

---

9. Error Response Format

A good API should return a **consistent error format**.

Example:

```
{
   "detail": "User not found"
}
```

A custom application might use:

```
{
   "status": 404,
   "message": "User not found",
   "error": "USER_NOT_FOUND"
}
```

Consistency makes frontend development easier.

Module 11 — Dependency Injection

Very important FastAPI interview topic.

1. What is Dependency Injection?

Dependency Injection means:

A function receives the resources or services it needs from an external source instead of creating them itself.

Simple example:

```
def get_database():
   return database_connection
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_database)):
   ...
```

FastAPI automatically calls get_database() and provides its result to db.

---

2. Why Dependency Injection?

Dependency Injection helps with:

- Code reuse
- Separation of concerns
- Authentication
- Database connections
- Common parameters
- Testing
- Cleaner code

Without dependency injection:

```
@app.get("/users")
def get_users():
    db = create_database_connection()
```

With dependency injection:

```
@app.get("/users")
def get_users(db = Depends(get_database)):
    ...
```

The endpoint doesn't need to know how the database connection is created.

---

3. Depends()

Depends() tells FastAPI:

"This parameter should be provided by another dependency function."

Example:

```
from fastapi import Depends

def get_user():
    return {"name": "John"}

@app.get("/profile")
def profile(user = Depends(get_user)):
    return user
```

Flow:

```
Request
  ↓
Depends(get_user)
  ↓
get_user()
  ↓
user
  ↓
profile()
```

---

4. Dependency Functions

A dependency is usually a normal Python function.

```
def get_current_user():
    return {
        "id": 1,
        "name": "John"
    }
```

Use it:

```
@app.get("/profile")
def profile(user = Depends(get_current_user)):
    return user
```

---

5. Dependency with Parameters

Dependencies can themselves have parameters.

```
def get_user(user_id: int):
    return {
        "id": user_id
    }
```

Then:

```
@app.get("/users/{user_id}")
def get_user_data(
    user = Depends(get_user)
):
    return user
```

FastAPI resolves the dependency's parameters from the request when applicable.

---

6. Multiple Dependencies

An endpoint can have multiple dependencies.

```
@app.get("/admin")
def admin_page(
    user = Depends(get_current_user),
    db = Depends(get_database)
):
    return {"message": "Admin page"}
```

Here there are two dependencies:

get_current_user()
get_database()

---

7. Nested Dependencies

A dependency can depend on another dependency.

```
def get_database():
    return database


def get_current_user(
    db = Depends(get_database)
):
    return get_user_from_db(db)
```

Then:

```
@app.get("/profile")
def profile(
    user = Depends(get_current_user)
):
    return user
```

Flow:

Endpoint
  ↓

```
get_current_user
  ↓
get_database
```

This is called **nested dependency injection**.

---

8. Reusable Dependencies

The biggest advantage is that dependencies can be reused across multiple endpoints.

```
def get_current_user():
    ...
```

Use it in:

```
@app.get("/profile")
def profile(user = Depends(get_current_user)):
    ...

@app.get("/orders")
def orders(user = Depends(get_current_user)):
    ...

@app.get("/dashboard")
def dashboard(user = Depends(get_current_user)):
    ...
```

Write once, reuse many times.

---

9. Database Dependency

A common FastAPI pattern is creating a database session dependency.

Conceptually:

```
def get_db():
    db = create_database_session()

    try:
        yield db
    finally:
        db.close()
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
   return db.query(...)
```

Flow:

```
Request
  ↓
get_db()
  ↓
Database Session
  ↓
Endpoint
  ↓
Response
  ↓
Session Cleanup
```

Interview Point

Database dependencies are commonly used to create, provide, and clean up database sessions.

---

10. Authentication Dependency

Authentication is another major use case.

```
def get_current_user():
   # verify token
   # identify user
   return user
```

Then:

```
@app.get("/profile")
def profile(
   user = Depends(get_current_user)
):
   return user
```

Every protected endpoint can reuse the same authentication dependency.

---

11. Dependency Overrides

Dependency overrides are mainly useful for **testing**.

Suppose production uses:

```
def get_database():
    return real_database
```

During testing, you can replace it:

```
app.dependency_overrides[get_database] = get_test_database
```

Now FastAPI uses the test dependency instead.

Interview Point

Dependency overrides allow us to replace dependencies, especially during testing.

---

12. Dependency Lifecycle

A dependency can have setup and cleanup logic.

The common pattern uses yield:

```
def get_db():
    db = create_session()

    try:
        yield db
    finally:
        db.close()
```

Think:

```
Setup
  ↓
yield resource
  ↓
Endpoint uses resource
  ↓
Cleanup
```

Module 12 — Routers

1. Why Use Routers?

As an application grows, putting every endpoint in main.py becomes difficult to maintain.

Instead of:

```
main.py
 ├── users
 ├── products
 ├── orders
 ├── payments
 ├── authentication
 └── ...
```

we separate routes:

main.py

```
routers/
 ├── users.py
 ├── products.py
 └── orders.py
```

Main Benefit

Routers help organize large FastAPI applications into smaller, maintainable modules.

---

2. APIRouter

APIRouter is used to create groups of related API routes.

Example:

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/users")
def get_users():
    return {"users": []}
```

---

3. Creating Separate Route Files

Example:

```
app/
├── main.py
└── routers/
    ├── users.py
    ├── products.py
    └── orders.py
```

users.py:

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/")
def get_users():
    return {"message": "Users"}
```

This keeps user-related endpoints together.

---

4. include_router()

include_router() connects a router to the main FastAPI application.

main.py:

```
from fastapi import FastAPI
from routers import users

app = FastAPI()

app.include_router(users.router)
```

Now routes defined inside users.py become part of the application.

---

5. Router Prefixes

A prefix adds a common path to all routes in a router.

```
app.include_router(
    users.router,
```

```
    prefix="/users"
)
```

If router contains:

```
@router.get("/")
```

Final endpoint becomes:

GET /users/

Another example:

```
app.include_router(
    products.router,
    prefix="/products"
)
```

---

6. Router Tags

Tags group endpoints in Swagger documentation.

```
app.include_router(
    users.router,
    prefix="/users",
    tags=["Users"]
)
```

In /docs, these endpoints appear under:

Users

---

7. Router Dependencies

Dependencies can be applied to an entire router.

```
app.include_router(
    admin.router,
    prefix="/admin",
    dependencies=[Depends(get_current_user)]
)
```

Now the dependency applies to routes in that router.

Useful for:

- Authentication
- Authorization
- Common checks

---

8. Organizing Large Applications

A common project structure:

```
app/
├── main.py
├── routers/
│   ├── users.py
│   ├── products.py
│   └── orders.py
├── models/
├── schemas/
├── services/
└── database/
```

Typical responsibility:

Folder	Purpose
main.py	Application entry point
routers/	API endpoints
models/	Database models
schemas/	Pydantic schemas
services/	Business logic
database/	DB configuration/session

Interview Point

Routers separate API routes by feature and improve scalability and maintainability.

Module 13 — Middleware

1. What is Middleware?

Middleware is code that runs **between the incoming request and the outgoing response**.

Conceptually:

Client
↓
Request
↓
Middleware
↓
FastAPI Endpoint
↓
Middleware
↓
Response
↓
Client

Middleware can inspect or modify requests and responses.

---

2. Request Middleware

Request middleware runs before the request reaches the endpoint.

Example use cases:

- Logging
- Authentication checks
- Request timing
- Adding request information

---

3. Response Middleware

After the endpoint produces a response, middleware can process the response before it goes back to the client.

Conceptually:

Request
↓
Middleware
↓
Endpoint
↓
Response
↓

Middleware
↓
Client

---

4. @app.middleware("http")

FastAPI allows custom HTTP middleware using:

```
@app.middleware("http")
async def my_middleware(request, call_next):
    response = await call_next(request)
    return response
```

call_next(request) passes the request to the next middleware or endpoint.

---

5. Middleware Execution Flow

Think of middleware like layers:

Client
↓
Middleware A
↓
Middleware B
↓
Endpoint
↑
Middleware B
↑
Middleware A
↑
Client

Request goes **down** through middleware.

Response comes **back up** through middleware.

---

6. Custom Middleware

Example:

```
@app.middleware("http")
async def custom_middleware(request, call_next):

    print("Request received")

    response = await call_next(request)

    print("Response generated")

    return response
```

---

7. Logging Middleware

Middleware can log:

HTTP method
URL
Status code
Processing time

Example:

```
@app.middleware("http")
async def logging_middleware(request, call_next):

    print(request.method, request.url)

    response = await call_next(request)

    print(response.status_code)

    return response
```

---

8. Timing Middleware

Used to measure API execution time.

Conceptually:

```
start = time.time()

response = await call_next(request)

duration = time.time() - start
```

Then log:

GET /users → 0.025 seconds

Useful for performance monitoring.

---

9. Authentication Middleware

Middleware can inspect requests and perform common authentication-related processing.

However, in FastAPI, **dependencies are often a better fit for endpoint-specific authentication/authorization**, because dependencies integrate naturally with route handling and can provide the authenticated user to the endpoint.

Interview Point

Middleware is suitable for cross-cutting concerns, while dependencies are often better for route-specific authentication and authorization.

---

10. CORS Middleware

FastAPI provides:

CORSMiddleware

Example:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This allows a frontend running on the specified origin to communicate with the FastAPI backend, subject to the configured CORS rules.

Module 14 — CORS

Very important for real-world FastAPI projects.

1. What is CORS?

CORS stands for:

Cross-Origin Resource Sharing

It is a browser security mechanism that controls whether a web page from one **origin** can make requests to a server at another origin.

Example:

Frontend:
http://localhost:3000

Backend:
http://localhost:8000

These are different origins because their ports differ.

The browser applies CORS rules when the frontend tries to call the backend.

---

2. Why CORS Occurs?

Suppose:

React Frontend
localhost:3000
↓
FastAPI Backend
localhost:8000

The browser sees different origins.

Without appropriate CORS configuration, the browser may block the frontend from accessing the backend response.

Important

CORS is primarily a **browser security mechanism**.

It is not simply a FastAPI error.

---

3. Browser Security

Browsers restrict cross-origin requests to protect users from malicious websites accessing resources they should not access.

The server can tell the browser which origins are allowed through CORS response headers.

---

4. CORSMiddleware

FastAPI uses Starlette's CORSMiddleware.

from fastapi.middleware.cors import CORSMiddleware

Example:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

---

5. Allowed Origins

An **origin** consists of:

scheme + host + port

Example:

http://localhost:3000

You can specify allowed frontend origins:

```
allow_origins=[
    "http://localhost:3000",
    "https://myfrontend.com"
]
```

Development

You may temporarily use:

allow_origins=["*"]

But in production, it's generally better to specify trusted origins.

---

6. Allowed Methods

You can specify which HTTP methods are allowed:

```
allow_methods=[
   "GET",
   "POST",
   "PUT",
   "DELETE"
]
```

Or:

allow_methods=["*"]

Common methods:

GET
POST
PUT
PATCH
DELETE

---

7. Allowed Headers

Headers sent by the frontend can also be controlled.

```
allow_headers=[
   "Content-Type",
   "Authorization"
]
```

Or:

allow_headers=["*"]

Common examples:

Content-Type
Authorization
Accept

---

8. Credentials

Credentials can include things such as:

- Cookies
- Browser authentication information
- Other credentialed cross-origin requests

FastAPI:

allow_credentials=True

Example:

```
app.add_middleware(
   CORSMiddleware,
   allow_origins=["http://localhost:3000"],
   allow_credentials=True,
   allow_methods=["*"],
   allow_headers=["*"],
)
```

Important

When credentials are enabled, you should configure trusted origins appropriately rather than relying on an unrestricted wildcard origin.

---

9. Frontend + FastAPI Communication

Typical architecture:

```
React
localhost:3000
     |
     | HTTP Request
     ↓
FastAPI
localhost:8000
     |
```

↓
Database

If frontend and backend have different origins:

Browser
↓
CORS Check
↓
FastAPI

FastAPI's CORS middleware sends the appropriate headers so the browser knows whether the frontend is allowed to access the response.

Module 15 — Authentication & Authorization

1. Authentication vs Authorization

These two are frequently asked in interviews.

Authentication

Authentication = Who are you?

It verifies the identity of the user.

Example:

Username + Password
↓
Verify User
↓
Authenticated

Authorization

Authorization = What are you allowed to do?

It checks the permissions of an authenticated user.

Example:

Admin → Can delete users
User → Cannot delete users

Easy Memory Trick

Authentication → WHO?
Authorization → WHAT CAN YOU DO?

---

2. User Registration

Registration means creating a new user account.

Typical flow:

User
↓
Registration API
↓
Validate data
↓
Check user already exists
↓
Hash password
↓
Save user in database
↓
Return success

Example data:

```
{
   "username": "john",
   "password": "mypassword"
}
```

Important

Never store plain-text passwords in the database.

---

3. Password Hashing

Password hashing converts a password into a one-way hash.

Password
↓
Hash Function

↓
Password Hash
↓
Database

Example:

mypassword
↓
$2b$12$....

You store the hash, not:

mypassword

Common password hashing approaches/libraries include:

- bcrypt
- Argon2

Interview Point

Password hashing protects passwords by storing a one-way representation instead of the original password.

---

4. Password Verification

During login:

Entered Password
↓
Hash Verification
↓
Stored Password Hash

The application checks whether the entered password matches the stored hash.

Conceptually:

verify_password(plain_password, stored_hash)

If it matches → login succeeds.

---

5. JWT

JWT stands for:

JSON Web Token

It is commonly used to represent authentication information between a client and server.

Typical flow:

Login
↓
Verify Credentials
↓
Generate JWT
↓
Client receives JWT
↓
Client sends JWT with requests
↓
Server validates JWT

---

6. Access Token

An **access token** is used to access protected resources.

Example:

GET /profile
Authorization: Bearer <access_token>

Access tokens are usually short-lived.

Purpose

Access Token → Access protected APIs

---

7. Refresh Token

A refresh token is used to obtain a new access token after the access token expires.

Conceptually:

Access Token

↓
Expires
↓
Refresh Token
↓
New Access Token

Why?

It avoids forcing the user to log in again frequently.

Easy Difference

Access Token	Refresh Token
Access API	Get new access token
Usually short-lived	Usually longer-lived
Sent frequently	Used less frequently

---

8. OAuth2

OAuth2 is an **authorization framework** used for delegated access.

It defines ways for applications to obtain access tokens.

FastAPI provides utilities that make implementing OAuth2-based authentication easier.

Interview Point

OAuth2 is an authorization framework, while JWT is a token format. They are related but are not the same thing.

---

9. OAuth2 Password Flow

The OAuth2 password flow is a flow where the client sends a username and password to the application's token endpoint.

Conceptually:

Username + Password
↓
Token Endpoint
↓

Verify Credentials
↓
Access Token

In modern applications, other OAuth2 flows are often preferred when appropriate, but the password flow is commonly taught and used in FastAPI examples.

---

10. OAuth2PasswordBearer

FastAPI provides:

OAuth2PasswordBearer

It is used to extract a bearer token from the Authorization header.

Example:

```
oauth2_scheme = OAuth2PasswordBearer(
   tokenUrl="token"
)
```

Then:

```
async def get_token(
   token: str = Depends(oauth2_scheme)
):
   return token
```

For:

Authorization: Bearer abc123

the dependency extracts:

abc123

Important

OAuth2PasswordBearer **does not itself validate the JWT**.

It extracts the bearer token. Your authentication logic must validate/decode the token.

---

11. OAuth2PasswordRequestForm

Used to receive username/password form data in the OAuth2 password flow.

Example:

```
from fastapi.security import OAuth2PasswordRequestForm

@app.post("/token")
async def login(
   form_data: OAuth2PasswordRequestForm = Depends()
):
   username = form_data.username
   password = form_data.password
```

The endpoint can then verify those credentials and issue a token.

---

12. Protected Routes

A protected route requires authentication.

Example concept:

```
@app.get("/profile")
async def profile(
   token: str = Depends(oauth2_scheme)
):
   return {"message": "Protected route"}
```

Flow:

```
Client
  ↓
Authorization: Bearer TOKEN
  ↓
FastAPI
  ↓
Extract Token
  ↓
Validate Token
  ↓
Get User
  ↓
Protected Endpoint
```

---

13. Role-Based Authorization

Role-based authorization controls access based on user roles.

Example:

Admin
Manager
User

Permissions:

Admin → Create + Read + Update + Delete
Manager → Read + Update
User → Read

Example:

```
if current_user.role != "admin":
    raise HTTPException(
        status_code=403,
        detail="Admin access required"
    )
```

---

14. Permissions

Permissions are specific actions a user can perform.

Examples:

users:read
users:create
users:update
users:delete

A user may have:

Role: Manager

Permissions:
- users:read
- users:update

Difference

Role → Group of access rules
Permission → Specific allowed action

---

15. Token Expiration

Tokens should generally have an expiration time.

Example:

Token created:
10:00 AM

Expires:
10:30 AM

After expiration:

Token → Invalid

The client may need to use a refresh token or authenticate again, depending on the system.

Module 16 — JWT Authentication

1. JWT Structure

A JWT normally contains three parts:

Header.Payload.Signature

Example conceptually:

xxxxx.yyyyy.zzzzz

Three parts:

Header
Payload
Signature

---

2. Header

The header contains information about the token.

For example:

```
{
   "alg": "HS256",
   "typ": "JWT"
}
```

Meaning:

alg → Signing algorithm
typ → Token type

---

3. Payload

The payload contains **claims**.

Example:

```
{
   "sub": "john",
   "role": "admin",
   "exp": 1787238000
}
```

Common claims:

sub → Subject/user identifier
exp → Expiration time
iat → Issued-at time

Important

Do not put sensitive secrets such as passwords inside a JWT payload.

JWT payload data is generally **encoded, not encrypted**.

---

4. Signature

The signature helps verify that the token was signed by a trusted party and has not been modified.

Conceptually:

Header + Payload
↓
Secret Key + Algorithm
↓
Signature

If someone modifies the payload:

Original Signature ≠ Calculated Signature

The token fails validation.

---

5. Secret Key

A secret key is used with certain signing algorithms to create and verify JWT signatures.

Example:

SECRET_KEY = "strong-secret-value"

In real projects:

Store secrets in environment variables or a secure secret manager, not directly in source code.

---

6. Algorithm

The algorithm defines how the JWT is signed.

A common symmetric algorithm is:

HS256

There are also asymmetric algorithms such as:

RS256

Simple understanding

Algorithm + Key
↓

JWT Signature

---

7. Token Expiration

JWT can contain an exp claim.

Example:

```
{
   "sub": "john",
   "exp": 1787238000
}
```

When the current time passes exp, the token should be rejected.

---

8. Creating JWT

Conceptually:

```
token = jwt.encode(
   payload,
   SECRET_KEY,
   algorithm="HS256"
)
```

Payload may contain:

```
{
   "sub": user.username,
   "exp": expiration_time
}
```

---

9. Decoding JWT

Conceptually:

```
payload = jwt.decode(
   token,
   SECRET_KEY,
   algorithms=["HS256"]
)
```

The JWT library verifies the signature and decodes the claims.

---

10. Validating JWT

JWT validation typically checks:

Token exists?
↓
Valid structure?
↓
Valid signature?
↓
Correct algorithm?
↓
Not expired?
↓
Valid claims?
↓
User exists/allowed?

Only after successful validation should the protected endpoint be allowed to continue.

---

11. Authorization: Bearer <token>

The standard HTTP header format is:

Authorization: Bearer <token>

Example:

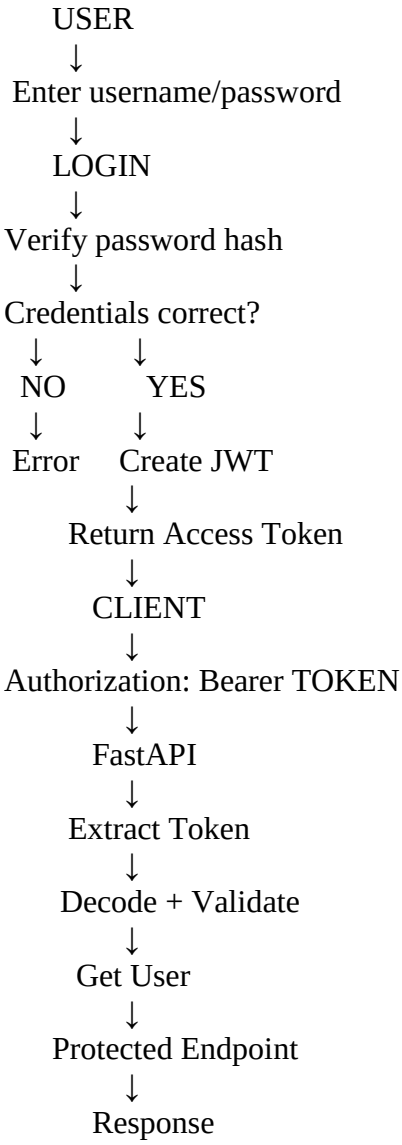
Authorization: Bearer eyJhbGciOiJIUzI1Ni...

The word:

Bearer

indicates that the token is being presented as a bearer credential.

Complete JWT Authentication Flow



Module 17 — Database Integration

1. Why Are Databases Needed?

Databases provide persistent storage for application data.

For example:

Users
Products
Orders
Payments

Vehicles
Bookings

Without a database, application data may be lost when the application restarts.

---

2. SQL Databases

SQL databases store data in tables.

Examples:

- PostgreSQL
- MySQL
- SQLite

Example:

users
---
id | name | email
---
1 | John | john@gmail.com
2 | Alex | alex@gmail.com

---

3. PostgreSQL

PostgreSQL is a powerful open-source relational database.

It is commonly used with production backend applications.

Advantages include:

- SQL support
- Transactions
- Relationships
- Constraints
- Good performance
- Advanced database features

---

4. MySQL

MySQL is another popular relational database.

It is widely used in:

- Web applications
- Enterprise systems
- Backend applications

---

5. SQLite

SQLite is a lightweight database stored in a file.

It is useful for:

- Learning
- Small applications
- Prototypes
- Testing

Example:

database.db

Unlike PostgreSQL/MySQL, it does not require a separate database server process.

---

6. ORM

ORM stands for:

Object-Relational Mapping

It allows developers to work with database records using programming-language objects instead of writing SQL for every operation.

Conceptually:

Python Object
↓
ORM
↓
Database Table

Example:

user = User(name="John")

The ORM can map this object to a database row.

---

7. SQLAlchemy

SQLAlchemy is a popular Python SQL toolkit and ORM.

FastAPI itself does not require SQLAlchemy, but SQLAlchemy is commonly used with FastAPI applications.

---

8. SQLModel

SQLModel is a library designed to combine Pydantic-style models with SQLAlchemy capabilities.

Conceptually:

Pydantic
+
SQLAlchemy
↓
SQLModel

It can reduce duplication in some FastAPI applications.

---

9. Database Connection

A database connection allows the application to communicate with the database.

Conceptually:

FastAPI
↓
Database Driver / ORM
↓
Database Connection
↓

PostgreSQL

The connection normally uses configuration such as:

Database URL
Username
Password
Host
Port
Database Name

Sensitive credentials should be stored securely, usually through environment variables or a secret manager.

---

10. Database Session

A session represents a unit of interaction with the database.

It is commonly used to:

- Query data
- Insert data
- Update data
- Delete data
- Commit transactions
- Roll back changes

Conceptually:

Request
↓
Create Session
↓
Perform DB Operations
↓
Commit/Rollback
↓
Close Session

---

11. CRUD Operations

CRUD means:

C → Create
R → Read
U → Update
D → Delete

CRUD	SQL
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

These are fundamental database operations in backend applications.

---

12. Transactions

A transaction is a group of database operations treated as one logical unit.

Example:

Create Order
+
Reduce Inventory
+
Create Payment Record

If something fails:

ROLLBACK

If everything succeeds:

COMMIT

Important Property

Transactions help maintain database consistency.

---

13. Connection Pooling

Connection pooling means maintaining a collection of reusable database connections.

Instead of:

Request
↓
Create new connection
↓
Use
↓
Destroy

the application can:

Connection Pool
├── Connection 1
├── Connection 2
├── Connection 3
└── Connection 4

Requests reuse available connections.

Benefit

It improves performance and reduces the overhead of repeatedly creating database connections.

---

14. Database Dependency

FastAPI dependencies are commonly used to provide database sessions.

```
def get_db():
    db = create_session()

    try:
        yield db
    finally:
        db.close()
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
    ...
```

This combines:

FastAPI Dependency Injection
+

Module 18 — SQLAlchemy with FastAPI

1. SQLAlchemy Introduction

SQLAlchemy is used to communicate with relational databases using Python.

It provides two major styles:

SQLAlchemy Core
SQLAlchemy ORM

For FastAPI applications, the ORM approach is commonly used.

---

2. Engine

The **Engine** manages communication between SQLAlchemy and the database.

Conceptually:

FastAPI
↓
SQLAlchemy Engine
↓
Database

Example:

engine = create_engine(DATABASE_URL)

The engine is generally created once for the application.

---

3. Session

A Session is used to interact with the database.

Example concept:

db = Session(engine)

Operations such as:

SELECT
INSERT
UPDATE
DELETE
COMMIT
ROLLBACK

are performed through the session/transaction machinery.

---

4. Base

In SQLAlchemy ORM, a declarative base is used to define ORM models.

Conceptually:

Base = declarative_base()

Models inherit from this base:

```
class User(Base):
    ...
```

Modern SQLAlchemy 2.x also provides newer typed declarative patterns, such as DeclarativeBase.

For interviews, understand:

Base provides the foundation for SQLAlchemy ORM model classes.

---

5. Models

A SQLAlchemy model represents a database table.

Example concept:

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)
```

```
    email = Column(String)
```

Conceptually:

Python Class → Database Table
Class Attribute → Database Column
Object → Database Row

---

6. Tables

A database table stores records in rows and columns.

Example:

```
users
--------------------------------
id | name | email
--------------------------------
1  | John | john@gmail.com
2  | Alex | alex@gmail.com
```

SQLAlchemy maps the Python model to this table.

---

7. Columns

Columns represent fields in a database table.

Example:

```
id = Column(Integer, primary_key=True)
name = Column(String)
email = Column(String)
```

Each represents a database column.

---

8. Primary Key

A primary key uniquely identifies each record.

Example:

id = Column(Integer, primary_key=True)

Values:

1
2
3
4

should uniquely identify rows.

Interview Point

A primary key uniquely identifies a row in a table.

---

9. Foreign Key

A foreign key creates a relationship between tables.

Example:

users
id
1

orders
id | user_id
10 | 1

orders.user_id references users.id.

Conceptually:

```
user_id = Column(
   ForeignKey("users.id")
)
```

---

10. Relationships

Relationships allow models to represent connections between database entities.

Example:

User
↓
Orders

A user may have multiple orders.

---

11. One-to-One

One record corresponds to one record.

Example:

User → UserProfile
One User
↓
One Profile

---

12. One-to-Many

One record can have many related records.

Example:

User
↓
Order 1
Order 2
Order 3

One user → many orders.

This is very common in real applications.

---

13. Many-to-Many

Many records can relate to many other records.

Example:

Students ↔ Courses

One student can take many courses.

One course can have many students.

Usually this requires an intermediate/junction table.

students
courses
student_courses

---

14. Queries

SQLAlchemy allows you to retrieve data using ORM/query APIs.

Conceptually:

SELECT users
WHERE id = 1

The exact syntax depends on the SQLAlchemy version/API style.

Modern SQLAlchemy commonly uses constructs such as:

select(User)

and executes them through a session.

---

15. Insert

Create a new object:

```
user = User(
    name="John",
    email="john@gmail.com"
)

db.add(user)
db.commit()
db.refresh(user)
```

Flow:

Create Object

↓
add()
↓
commit()
↓
Database

---

16. Select

Retrieve records.

Modern SQLAlchemy style:

```
result = db.execute(
    select(User)
)

users = result.scalars().all()
```

Conceptually:

```
select()
  ↓
execute()
  ↓
scalars()
  ↓
all()
```

---

17. Update

Typical flow:

```
Find record
  ↓
Change values
  ↓
Commit
```

Example:

```
user.name = "Alex"
db.commit()
db.refresh(user)
```

---

18. Delete

Typical flow:

```
db.delete(user)
db.commit()
```

Flow:

```
Find Object
  ↓
delete()
  ↓
commit()
```

---

19. Transactions

A transaction groups database operations into one logical unit.

Example:

```
Operation 1
Operation 2
Operation 3
  ↓
All succeed → COMMIT
Any failure → ROLLBACK
```

SQLAlchemy sessions are commonly used together with transaction management.

---

20. Session Management

A FastAPI application should manage sessions carefully.

Common pattern:

```
def get_db():
    db = SessionLocal()

    try:
        yield db
```

```
    finally:
        db.close()
```

Then:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
    ...
```

This ensures the session is cleaned up after the request.

Module 20 — Async Programming

Frequently asked in FastAPI interviews.

1. Synchronous Programming

In synchronous programming, tasks generally execute sequentially.

Example:

Task A
↓
Wait
↓
Task B
↓
Wait
↓
Task C

If Task A is waiting for I/O, execution may not move to another task in the same synchronous flow.

---

2. Asynchronous Programming

Asynchronous programming allows an application to work on other tasks while waiting for an I/O operation to complete.

Example:

Task A

↓
Waiting for DB
↘
Task B executes
↓
Task A completes

This is especially useful for I/O-heavy applications.

Examples of I/O:

- Database queries
- HTTP requests
- File/network operations

---

3. async

async defines an asynchronous function.

```
async def get_users():
    ...
```

An async def function returns a coroutine when called.

---

4. await

await pauses the current coroutine until an awaitable operation completes, allowing the event loop to run other work.

Example:

```
async def get_data():
    result = await some_async_operation()
    return result
```

Easy Memory

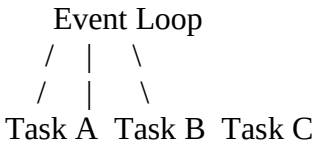
async → defines async function
await → waits for async operation

---

5. Event Loop

The event loop is the mechanism that coordinates asynchronous tasks.

Conceptually:



When one task is waiting for I/O, the event loop can run another ready task.

---

6. Concurrency

Concurrency means multiple tasks can **make progress during overlapping periods**.

It does not necessarily mean that tasks execute simultaneously on different CPU cores.

For FastAPI, async concurrency is particularly useful for I/O-bound workloads.

---

7. Blocking vs Non-Blocking

Blocking

A blocking operation prevents the current execution flow from making progress until it completes.

Example:

time.sleep(5)

Non-blocking / Async I/O

An async operation can yield control while waiting.

Example concept:

await asyncio.sleep(5)

This allows other asynchronous work to run during the wait.

---

8. Async Endpoints

FastAPI supports:

```
@app.get("/users")
async def get_users():
    data = await get_data()
    return data
```

This is useful when the endpoint performs asynchronous I/O.

---

9. When to Use async def

Use async def when you are working with **async-compatible I/O libraries** and need asynchronous operations.

Examples:

Async database driver
Async HTTP client
Async file/network operations

Example:

```
@app.get("/users")
async def get_users():
    users = await async_database_call()
    return users
```

---

10. When Normal def Is Better

Use normal:

def

when your endpoint or dependency is synchronous.

FastAPI can run synchronous path operation functions in a thread pool so they don't directly block the event loop.

Example:

```
@app.get("/users")
```

```
def get_users():
    return database.query(...)
```

This is perfectly valid when using a synchronous database library/ORM.

Important Interview Point

Don't use async def just because FastAPI supports it. Use it when the operations you perform are compatible with asynchronous execution.

---

11. Async Database Operations

For asynchronous database access, you need an async-compatible database driver/library.

Conceptually:

```
FastAPI async endpoint
        ↓
Async DB Session
        ↓
Async DB Driver
        ↓
Database
```

Example concept:

```
result = await session.execute(...)
```

The exact implementation depends on the database and library being used.

---

12. Async HTTP Requests

If your FastAPI endpoint calls another API, an async HTTP client can be useful.

Conceptually:

```
FastAPI
  ↓
await HTTP Request
  ↓
External API
  ↓
```

Response

While waiting for the external API, the event loop can work on other requests.

Important: Async Does NOT Automatically Mean Faster

This is a common interview trap.

async is especially useful for:

I/O-bound workloads

It does not automatically make CPU-heavy operations faster.

For example:

Large ML computation
Complex image processing
Heavy mathematical calculation

may require different approaches such as multiprocessing, worker processes, or specialized compute resources.